

Google Authentication Helper

Step-by-Step Explanation

A beginner-friendly guide to service-account JSON, environment variables, Secret Manager, credential scopes, project detection, and Application Default Credentials

Purpose of this file: It gives the rest of the project one consistent function for obtaining Google credentials. The same project can therefore authenticate differently on a laptop, in Airflow, or in Google Cloud without rewriting every script.

Prepared as a learning and revision guide

1. What the complete file does

This module does not call BigQuery, Google Sheets, Google Drive, or Secret Manager for business data. Its job is authentication: it locates identity information and turns it into a credentials object that other Google libraries can use.

1. Looks for raw service-account JSON stored directly in the `SERVICE_ACCOUNT_JSON` environment variable.
2. Looks for a service-account file whose path is stored in `GOOGLE_APPLICATION_CREDENTIALS`.
3. Looks for a standard local file named `credentials/service_account.json`.
4. Optionally downloads service-account JSON from Google Secret Manager.
5. Creates a Google Credentials object with the requested OAuth scopes.
6. Determines the GCP project ID from environment variables, the JSON content, or Google automatic detection.
7. Falls back to Application Default Credentials when the custom service-account sources are not available.

Two different stages: `load_service_account_info()` finds a Python dictionary containing service-account information. `load_google_credentials()` converts that dictionary into an authentication object that Google client libraries can use.

Credential search priority

Priority	Credential source	When it is useful
1	<code>SERVICE_ACCOUNT_JSON</code>	A platform injects the entire JSON document as an environment variable.
2	<code>GOOGLE_APPLICATION_CREDENTIALS</code>	A local or server environment stores the key in a file and supplies its path.
3	<code>credentials/service_account.json</code>	A predictable local development file is present inside the project structure.
4	Google Secret Manager	The JSON document is securely stored as a secret and downloaded at runtime.
5	Application Default Credentials	Google automatically identifies the current user or the service account attached to the runtime.

2. The complete script

The following is the complete helper module. The remaining pages explain it section by section.

```
import json
import os
from pathlib import Path
from typing import Any, Tuple

import google.auth
from google.oauth2.service_account import Credentials

def _read_json_file(path: Path) -> dict[str, Any]:
    with path.open("r", encoding="utf-8") as handle:
        return json.load(handle)

def _get_default_service_account_file() -> Path:
    return Path(__file__).resolve().parents[1] / "credentials" / "service_account.json"

def load_service_account_info() -> Tuple[dict[str, Any], str]:
    raw_json = os.getenv("SERVICE_ACCOUNT_JSON")
    if raw_json:
        try:
            return json.loads(raw_json), "SERVICE_ACCOUNT_JSON"
        except json.JSONDecodeError as exc:
            raise RuntimeError("SERVICE_ACCOUNT_JSON is not valid JSON.") from exc

    env_path = os.getenv("GOOGLE_APPLICATION_CREDENTIALS")
    if env_path:
        credential_file = Path(env_path).expanduser()
        if credential_file.exists():
            return _read_json_file(credential_file), f"GOOGLE_APPLICATION_CREDENTIALS:{credential_file}"

    default_path = _get_default_service_account_file()
    if default_path.exists():
        return _read_json_file(default_path), f"credentials/service_account.json"

    project_id = os.getenv("GCP_PROJECT_ID") or os.getenv("GOOGLE_CLOUD_PROJECT")
    secret_name = os.getenv("GOOGLE_SECRET_NAME") or os.getenv("SERVICE_ACCOUNT_SECRET_NAME")
    if project_id and secret_name:
        try:
            from google.cloud import secretmanager
        except ImportError as exc:
            raise RuntimeError(
                "google-cloud-secret-manager is not installed. Install it with: pip install google-cloud-secret-manager"
            ) from exc

        client = secretmanager.SecretManagerServiceClient()
        name = f"projects/{project_id}/secrets/{secret_name}/versions/latest"
        response = client.access_secret_version(request={"name": name})
        payload = response.payload.data.decode("utf-8")
        return json.loads(payload), f"Secret Manager:{secret_name}"

    raise FileNotFoundError(
        "No service account credentials were found. Provide one of the following:\n"
        "1. SERVICE_ACCOUNT_JSON environment variable with raw JSON\n"
        "2. GOOGLE_APPLICATION_CREDENTIALS pointing to a service account JSON file\n"
        "3. credentials/service_account.json\n"
        "4. GCP_PROJECT_ID and GOOGLE_SECRET_NAME for Secret Manager"
    )

def load_google_credentials(scopes: list[str]) -> tuple[Any, str, str | None]:
    try:
        service_account_info, source = load_service_account_info()
        credentials = Credentials.from_service_account_info(service_account_info, scopes=scopes)
        project_id = (
            os.getenv("GCP_PROJECT_ID")
            or os.getenv("GOOGLE_CLOUD_PROJECT")
            or service_account_info.get("project_id")
        )
        return credentials, source, project_id
    except FileNotFoundError:
        credentials, detected_project_id = google.auth.default(scopes=scopes)
        project_id = os.getenv("GCP_PROJECT_ID") or os.getenv("GOOGLE_CLOUD_PROJECT") or detected_project_id
        return credentials, "Application Default Credentials", project_id
```

3. Import statements

Import	Simple meaning	Why it is needed here
<code>import json</code>	Works with JSON text and Python dictionaries.	Service-account credentials are represented as JSON.
<code>import os</code>	Reads operating-system environment variables.	The code obtains credential settings and project settings without hard-coding them.
<code>from pathlib import Path</code>	Provides safe, readable file-path operations.	The helper opens credential files and builds a default path that works across operating systems.
<code>from typing import Any, Tuple</code>	Provides type-hint building blocks.	The JSON dictionary can contain mixed value types, and one function returns two values.
<code>import google.auth</code>	Imports Google authentication discovery.	It provides <code>google.auth.default()</code> , the final fallback.
<code>from google.oauth2.service_account import Credentials</code>	Imports the service-account credential class.	It converts a service-account dictionary into a usable Google credentials object.

Understanding the type hints

```
dict[str, Any]
Tuple[dict[str, Any], str]
tuple[Any, str, str | None]
```

- `dict[str, Any]` means the keys are strings, while values may be strings, numbers, lists, dictionaries, or other types.
- `Tuple[dict[str, Any], str]` means the function returns exactly two values: a dictionary and a source-description string.
- `tuple[Any, str, str | None]` means the function returns three values. The project ID may be a string or `None` when no project can be determined.
- `Tuple[...]` and `tuple[...]` express the same general idea here; the lowercase form is the newer built-in style.

4. Reading a JSON file safely

```
def _read_json_file(path: Path) -> dict[str, Any]:  
    with path.open("r", encoding="utf-8") as handle:  
        return json.load(handle)
```

Code	What happens
<code>def _read_json_file(path: Path)</code>	Defines a private helper that expects a Path object.
<code>-> dict[str, Any]</code>	Documents that the function returns a dictionary loaded from JSON.
<code>path.open("r", encoding="utf-8")</code>	Opens the file in read mode using UTF-8 character encoding.
<code>as handle</code>	Stores the open file object in a temporary variable named handle.
<code>with ...</code>	Automatically closes the file when the indented block finishes, including when an error occurs.
<code>json.load(handle)</code>	Reads JSON directly from the open file and converts it into Python objects.

json.load versus json.loads: `json.load(handle)` reads JSON from an open file. `json.loads(text)` reads JSON from a string already held in memory.

For a service-account file, the resulting dictionary commonly contains fields such as `project_id`, `client_email`, `private_key`, `token_uri`, and `private_key_id`. This helper does not validate every field itself; the Google Credentials class performs the credential-specific validation later.

5. Building the default credential-file path

```
def _get_default_service_account_file() -> Path:
    return Path(__file__).resolve().parents[1] / "credentials" / "service_account.json"
```

Part	Meaning
<code>__file__</code>	The path of the Python helper file that is currently running.
<code>Path(__file__)</code>	Converts that path into a Path object.
<code>.resolve()</code>	Produces an absolute, normalised path.
<code>.parents[1]</code>	Moves two directory levels upward from the helper file.
<code>/ "credentials"</code>	Appends a folder named credentials.
<code>/ "service_account.json"</code>	Appends the expected credential filename.

Path example

```
Suppose __file__ is:
D:\learning_airflow\dags\utils\google_auth.py

Path(__file__).resolve().parents[1] becomes:
D:\learning_airflow\dags

The returned default path becomes:
D:\learning_airflow\dags\credentials\service_account.json
```

Why use `__file__`?: The path is calculated relative to the helper module, not relative to whichever terminal folder happens to be active. This makes execution more predictable in Airflow.

6. load_service_account_info(): first credential source

```
raw_json = os.getenv("SERVICE_ACCOUNT_JSON")
if raw_json:
    try:
        return json.loads(raw_json), "SERVICE_ACCOUNT_JSON"
    except json.JSONDecodeError as exc:
        raise RuntimeError("SERVICE_ACCOUNT_JSON is not valid JSON.") from exc
```

8. `os.getenv("SERVICE_ACCOUNT_JSON")` searches for an environment variable with that exact name.
9. When the variable does not exist, `os.getenv` returns `None`. The `if` block is skipped.
10. When it exists and contains text, `json.loads(raw_json)` converts that text into a Python dictionary.
11. The function immediately returns two values: the dictionary and the label `SERVICE_ACCOUNT_JSON`.
12. Because the function returns immediately, lower-priority credential sources are not checked.

Why the try/except is important

An environment variable is plain text. It may be incomplete, incorrectly quoted, or contain malformed JSON. `json.loads` raises `json.JSONDecodeError` when the text cannot be parsed.

```
except json.JSONDecodeError as exc:
    raise RuntimeError("SERVICE_ACCOUNT_JSON is not valid JSON.") from exc
```

- The user receives a clearer, project-specific error message.
- `from exc` keeps the original JSON parsing error as the underlying cause.
- The function does not silently ignore a configured but broken credential source. This makes configuration problems easier to find.

7. Second and third credential sources

Source 2: GOOGLE_APPLICATION_CREDENTIALS

```
env_path = os.getenv("GOOGLE_APPLICATION_CREDENTIALS")
if env_path:
    credential_file = Path(env_path).expanduser()
    if credential_file.exists():
        return _read_json_file(credential_file), f"GOOGLE_APPLICATION_CREDENTIALS:
{credential_file}"
```

13. The environment variable stores a file path, not the complete JSON document.
14. `Path(env_path)` converts the text path into a `Path` object.
15. `expanduser()` converts a leading `~` into the current user home directory when that notation is used.
16. `exists()` checks whether the target path is present before trying to open it.
17. `_read_json_file()` reads and converts the file into a dictionary.
18. The returned source string includes the actual file path, which is useful for debugging.

Important behaviour: If `GOOGLE_APPLICATION_CREDENTIALS` is set but the file does not exist, this custom function does not fail immediately. It continues to the next source.

Source 3: project default file

```
default_path = _get_default_service_account_file()
if default_path.exists():
    return _read_json_file(default_path), "credentials/service_account.json"
```

This source supports a predictable local project layout. The `f` prefix in the original return value is unnecessary because the string contains no replacement expression, but it does not cause a problem.

8. Reading credentials from Secret Manager

```
project_id = os.getenv("GCP_PROJECT_ID") or os.getenv("GOOGLE_CLOUD_PROJECT")
secret_name = os.getenv("GOOGLE_SECRET_NAME") or os.getenv("SERVICE_ACCOUNT_SECRET_NAME")
```

The or operator selects the first non-empty value. This permits two possible environment-variable names for the project and two possible names for the secret.

Setting needed	Primary variable	Alternative variable
Project ID	GCP_PROJECT_ID	GOOGLE_CLOUD_PROJECT
Secret name	GOOGLE_SECRET_NAME	SERVICE_ACCOUNT_SECRET_NAME

```
if project_id and secret_name:
    try:
        from google.cloud import secretmanager
    except ImportError as exc:
        raise RuntimeError(
            "google-cloud-secret-manager is not installed. Install it with: pip install google-cloud-secret-manager"
        ) from exc
```

- Both the project ID and secret name must exist before this branch can run.
- The Secret Manager package is imported only when it is needed. This is called a lazy import.
- If the optional package is missing, the code raises an installation message rather than an unclear ImportError.

```
client = secretmanager.SecretManagerServiceClient()
name = f"projects/{project_id}/secrets/{secret_name}/versions/latest"
response = client.access_secret_version(request={"name": name})
payload = response.payload.data.decode("utf-8")
return json.loads(payload), f"Secret Manager:{secret_name}"
```

9. Secret Manager lines in detail

Line	Explanation
<code>SecretManagerServiceClient()</code>	Creates a client that can communicate with the Secret Manager API. The client itself must also authenticate, usually through the environment's available Google credentials.
<code>projects/{project_id}/secrets/{secret_name}/versions/latest</code>	Builds the full resource name. latest requests the version currently represented by the latest alias.
<code>access_secret_version(...)</code>	Requests access to the selected secret version. IAM permission and an enabled Secret Manager API are still required.
<code>response.payload.data</code>	Obtains the secret payload as bytes.
<code>.decode("utf-8")</code>	Converts the bytes into a normal Python string.
<code>json.loads(payload)</code>	Converts the JSON text stored in the secret into a Python dictionary.
<code>"Secret Manager:{secret_name}"</code>	Returns a safe source description. It reports the secret name, not the private key content.

Authentication dependency: To download a service-account key from Secret Manager, the Secret Manager client first needs some existing identity with permission to access that secret. In Google Cloud, this is commonly the service account attached to the running environment.

Do not print the payload: The decoded payload contains sensitive private-key material. The program may report the source label, but it should never log `service_account_info`, `payload`, `private_key`, or the complete environment variable.

10. When no custom service-account source is found

```
raise FileNotFoundError(
    "No service account credentials were found. Provide one of the following:\n"
    "1. SERVICE_ACCOUNT_JSON environment variable with raw JSON\n"
    "2. GOOGLE_APPLICATION_CREDENTIALS pointing to a service account JSON file\n"
    "3. credentials/service_account.json\n"
    "4. GCP_PROJECT_ID and GOOGLE_SECRET_NAME for Secret Manager"
)
```

This exception has a special purpose in the complete design. It means the custom loader could not locate any of its four service-account sources. `load_google_credentials()` catches this exact exception and then tries Application Default Credentials.

Situation	What happens
No custom source is configured	<code>FileNotFoundError</code> is raised and then caught by <code>load_google_credentials()</code> . ADC is attempted.
<code>SERVICE_ACCOUNT_JSON</code> contains invalid JSON	<code>RuntimeError</code> is raised. The code does not fall back, because a supplied credential source is broken and should be corrected.
Secret Manager package is missing	<code>RuntimeError</code> is raised with an installation instruction.
Secret Manager denies access	The Google API error propagates. It is not converted into <code>FileNotFoundError</code> .
A JSON file exists but is invalid	The JSON parsing error propagates instead of silently selecting another identity.

Why catch only `FileNotFoundError`?: The fallback is intended for absence of custom credentials, not for hiding invalid JSON, missing packages, incorrect IAM, or other real configuration failures.

11. Creating the usable Google credentials object

```
def load_google_credentials(scopes: list[str]) -> tuple[Any, str, str | None]:
    try:
        service_account_info, source = load_service_account_info()
        credentials = Credentials.from_service_account_info(service_account_info, scopes=scopes)
```

19. The caller passes a list of OAuth scope strings.
20. `load_service_account_info()` returns two values, which are unpacked into `service_account_info` and `source`.
21. `Credentials.from_service_account_info()` reads fields such as the client email and private key from the dictionary.
22. The scopes are attached to the credential object so it can request tokens for the intended Google APIs.
23. The result is an object that can be supplied to libraries such as BigQuery, Google Sheets, Google Drive, or Secret Manager clients.

What scopes do and do not do

- A scope describes the API access requested by the token.
- A scope does not grant IAM roles by itself.
- The service account or user still needs the required permissions on the target project, dataset, spreadsheet, secret, or other resource.

```
example_scopes = [
    "https://www.googleapis.com/auth/cloud-platform",
    "https://www.googleapis.com/auth/spreadsheets",
    "https://www.googleapis.com/auth/drive",
]

credentials, source, project_id = load_google_credentials(example_scopes)
```

12. Determining the project ID

```
project_id = (
    os.getenv("GCP_PROJECT_ID")
    or os.getenv("GOOGLE_CLOUD_PROJECT")
    or service_account_info.get("project_id")
)
```

The project ID is selected in priority order. Python evaluates the expressions from top to bottom and stops at the first non-empty value.

Priority	Source	Reason
1	GCP_PROJECT_ID	Allows the application to explicitly override the project.
2	GOOGLE_CLOUD_PROJECT	Uses the standard project variable when the custom variable is absent.
3	service_account_info.get("project_id")	Uses the project ID stored inside the service-account JSON.

Why .get()?: service_account_info["project_id"] would raise KeyError when the key is absent. .get("project_id") safely returns None instead.

The function returns credentials, source, and project_id. Returning the source is useful for operational logs such as "credentials loaded from Application Default Credentials" without exposing secret content.

```
return credentials, source, project_id

# Possible result conceptually:
(
    <google.oauth2.service_account.Credentials object>,
    "Secret Manager:learning-service-account-key",
    "halogen-project-id"
)
```

13. Application Default Credentials fallback

```
except FileNotFoundError:
    credentials, detected_project_id = google.auth.default(scopes=scopes)
    project_id = os.getenv("GCP_PROJECT_ID") or os.getenv("GOOGLE_CLOUD_PROJECT") or
    detected_project_id
    return credentials, "Application Default Credentials", project_id
```

- 24. This block runs only when `load_service_account_info()` raises `FileNotFoundError`.
- 25. `google.auth.default(scopes=scopes)` asks the Google authentication library to discover credentials from the current environment.
- 26. It returns a credentials object and, when detectable, a project ID.
- 27. Environment variables still have priority over the automatically detected project ID.
- 28. The source label is set to Application Default Credentials.

Typical environments

Environment	Possible ADC identity
Local laptop	User credentials created for local application development, or another supported ADC configuration.
Cloud Composer / Airflow	The service account attached to the Composer environment or relevant Google Cloud runtime.
Cloud Run / Compute Engine	The service account attached to that deployed service or virtual machine.

Best fit for Composer: In a managed Google Cloud environment, ADC with a properly permissioned attached service account is usually cleaner than copying a long-lived private-key JSON file into the DAG project.

14. Complete execution flow

Step	Question asked by the code	Next action
1	Does SERVICE_ACCOUNT_JSON contain text?	Parse it. If valid, return it. If invalid, stop with RuntimeError.
2	Does GOOGLE_APPLICATION_CREDENTIALS point to an existing file?	Read and return that file.
3	Does credentials/service_account.json exist?	Read and return the local file.
4	Are both project ID and secret name available?	Access the latest Secret Manager version, decode it, parse JSON, and return it.
5	Were none of the four sources found?	Raise FileNotFoundError.
6	Did load_google_credentials catch FileNotFoundError?	Call google.auth.default() to discover ADC.
7	Which project ID should be used?	Prefer GCP_PROJECT_ID, then GOOGLE_CLOUD_PROJECT, then JSON or detected project.
8	What is returned to the caller?	Credentials object, source description, and project ID.

In one sentence: The module tries explicit service-account methods first and uses Google automatic authentication only when no custom service-account source is found.

15. Practical setup examples

Example A: Windows environment-variable file path

```
set GOOGLE_APPLICATION_CREDENTIALS=D:\learning_airflow\credentials\service_account.json
set GCP_PROJECT_ID=halogen-project-id
```

The helper reads the path from the environment variable, checks that the file exists, and loads its JSON.

Example B: Secret Manager

```
set GCP_PROJECT_ID=halogen-project-id
set GOOGLE_SECRET_NAME=learning-service-account-key
```

The runtime identity must be able to access the selected secret. The secret payload must contain valid service-account JSON.

Example C: Application Default Credentials

```
# No custom credential variables or local JSON file are provided.
credentials, source, project_id = load_google_credentials(scopes)

# The source returned by the helper is:
"Application Default Credentials"
```

This is suitable when the local machine has ADC configured or when the Google Cloud runtime already has an attached service account.

16. How another script would use this helper

```
from google.cloud import bigquery
from utils.google_auth import load_google_credentials

SCOPES = ["https://www.googleapis.com/auth/cloud-platform"]

credentials, credential_source, project_id = load_google_credentials(SCOPES)

print(f"Credential source: {credential_source}")
print(f"Project ID: {project_id}")

client = bigquery.Client(
    project=project_id,
    credentials=credentials,
)

rows = client.query("SELECT CURRENT_DATE() AS run_date").result()
for row in rows:
    print(row.run_date)
```

- 29. The calling script defines the scopes it needs.
- 30. The helper returns a credentials object, a safe source label, and a project ID.
- 31. The BigQuery client receives those values.
- 32. The query is authenticated as the identity represented by the credentials object.

Source logging is safe only at the label level: Printing `credential_source` is useful. Printing the credential dictionary or private key is not safe.

17. Important behaviours and possible improvements

Observation	Why it matters
The function supports several credential methods.	The same codebase can run in different environments without changing the business scripts.
The source order is fixed.	An existing SERVICE_ACCOUNT_JSON value overrides all lower-priority sources.
A missing GOOGLE_APPLICATION_CREDENTIALS file is initially skipped.	The custom loader continues, but ADC may later inspect that variable again and report an authentication error.
The Secret Manager payload is parsed without a dedicated JSON error message.	Malformed secret JSON raises JSONDecodeError directly; a clearer RuntimeError could be added.
The local default path places a private key inside the project tree.	It must be excluded from Git, protected with file permissions, and preferably avoided in production.
The source string for the local file uses an unnecessary f prefix.	It is harmless but can be simplified to a normal string.
The return credential type is Any.	This is flexible because service-account and ADC credentials may be different concrete classes, though a common credentials base type could make the hint more precise.
Scopes are supplied by the caller.	Each script can request the scopes it needs, but scopes and IAM permissions must remain consistent.

A useful validation improvement

```
required_fields = {"project_id", "client_email", "private_key", "token_uri"}
missing = required_fields - service_account_info.keys()
if missing:
    raise RuntimeError(f"Service-account JSON is missing fields: {sorted(missing)}")
```

This optional check can produce an earlier, easier-to-understand message before constructing the Credentials object.

18. Security guidance

- Never commit service_account.json to GitHub. Add credentials/, *.json where appropriate, or the exact filename to .gitignore.
- Never paste the private key into logs, screenshots, email, documentation, or chat messages.
- Use the least-privileged IAM roles required by the application.
- Prefer short-lived or automatically supplied credentials, such as ADC and attached service accounts, over downloadable long-lived service-account keys when the environment supports them.
- When Secret Manager is used, grant access only to the runtime identity that requires the secret.
- Enable and use the relevant Google APIs, but remember that enabling an API does not grant IAM permission.
- Rotate, disable, and remove service-account keys that are no longer required.

For your Cloud Composer learning project: The clean production-style direction is to let Composer use its attached service account through ADC and grant that account the required BigQuery, Secret Manager, Drive, or Sheets permissions. A JSON key can be useful for learning or specific external environments, but it introduces additional key-management risk.

Suggested .gitignore entries

```
# Service-account credentials
credentials/
service_account.json
*.key.json

# Local environment values
.env
.env.*
```

19. Final summary

Function or item	Purpose
<code>_read_json_file()</code>	Reads JSON from a file and returns a Python dictionary.
<code>_get_default_service_account_file()</code>	Builds the expected local service-account file path relative to the helper module.
<code>load_service_account_info()</code>	Searches four explicit service-account sources and returns raw service-account information plus its source.
<code>Credentials.from_service_account_info()</code>	Converts raw service-account information into a Google credentials object with scopes.
<code>google.auth.default()</code>	Provides the final automatic credential-discovery fallback.
<code>source</code>	A human-readable label showing which authentication route was selected.
<code>project_id</code>	The GCP project that downstream client libraries should use; it may be None if it cannot be detected.
<code>scopes</code>	The Google API access requested for the generated authentication token.

Core understanding: This helper separates authentication setup from business logic. Your DAGs and Python scripts can focus on their actual work while one shared module handles credential discovery consistently.

Official documentation consulted

- Google Auth Python documentation: Application Default Credentials and service-account Credentials.
- Google Cloud Secret Manager documentation: accessing a secret version and using the latest version alias.
- Google Cloud IAM documentation: best practices for managing service-account keys.